

LECTURE

Understanding Terraform State

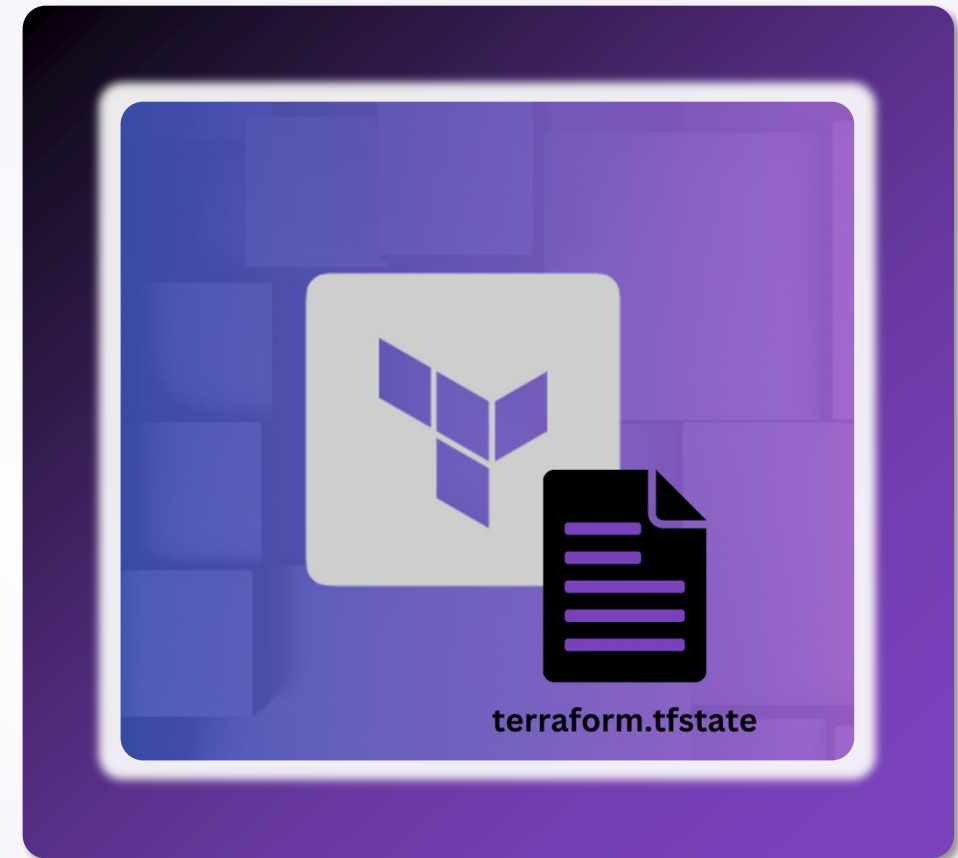




Introduction to Terraform State

Terraform state is a file that tracks the resources managed by Terraform, including their IDs and relationships.

- ✧ It acts as a single source of truth for your infrastructure. Without the state file, Terraform would not know its current state.
- ✧ Terraform cannot operate without state
- ✧ Before any operation, Terraform does a refresh to update the state with the real-world infrastructure

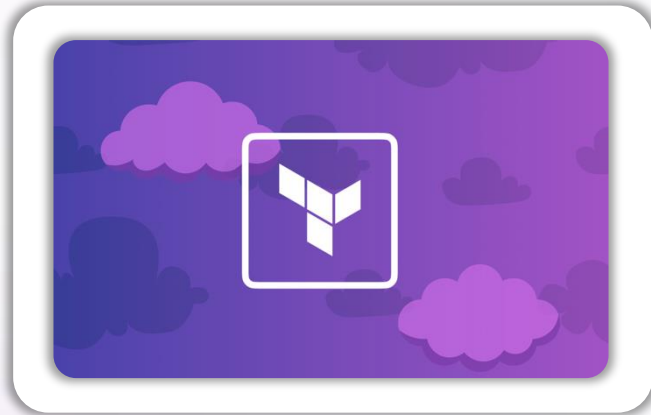


Why Terraform State is Important



Resource Management

Enables Terraform to determine the difference between the desired and current state.



Dependency Management

Tracks resource relationships to apply updates in the correct order.

Terraform does a refresh to update the state before any operations



Code Collaboration

Remote state enables secure, shared workflows for teams.

State File

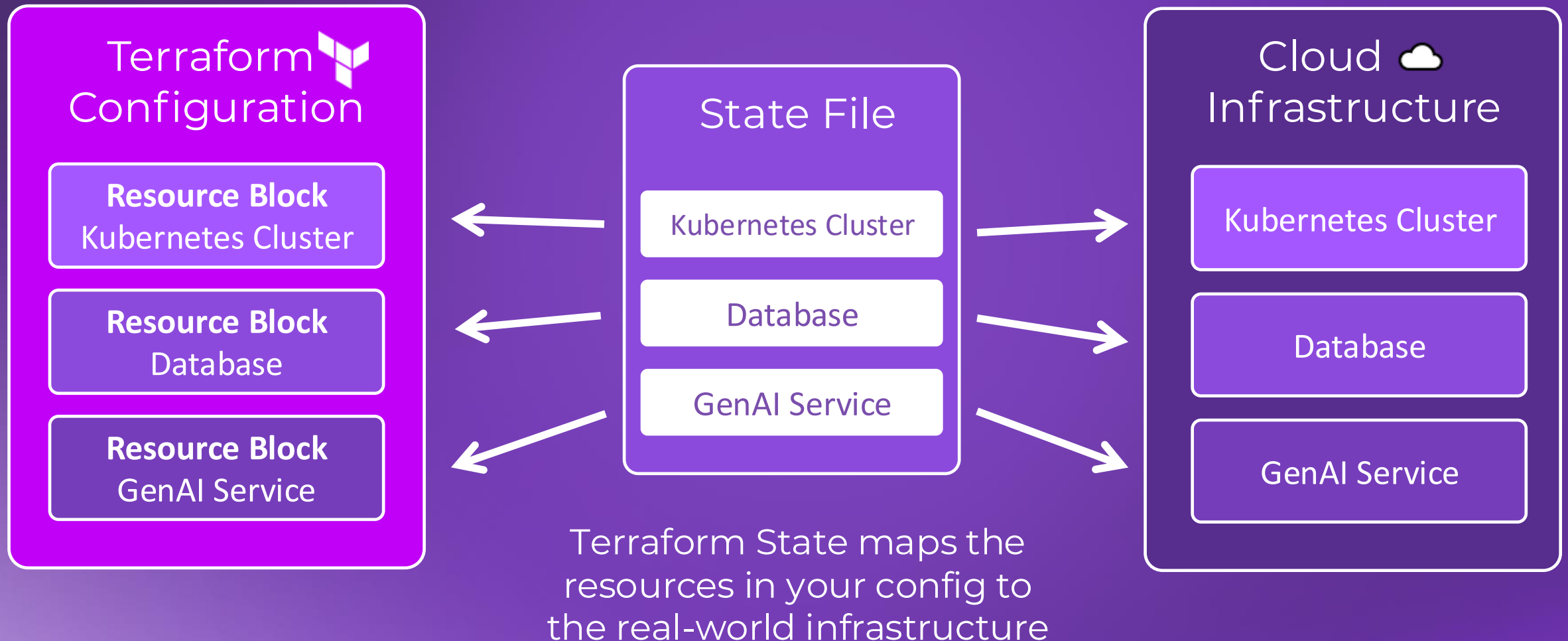


- ✧ Terraform stores state in file, which acts as a sort of database to map Terraform configurations to real-world resources
- ✧ Alongside the mapping between resources and remote objects, Terraform also tracks metadata for internal purposes.
- ✧ This metadata allows Terraform to track order of operations, the provider used, etc.

```
1  {
2    "module": "module.nomad-client[\"nomad-client-3\"]",
3    "mode": "managed",
4    "type": "aws_instance",
5    "name": "this",
6    "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
7    "instances": [
8      {
9        "index_key": 0,
10       "schema_version": 1,
11       "attributes": {
12         "ami": "ami-03a6eaae9938c858c",
13         "arn": "arn:aws:ec2:us-east-1:634211456147:instance/i-0eb4591447d4fdf7c",
14         "associate_public_ip_address": true,
15         "availability_zone": "us-east-1c",
16         "capacity_reservation_specification": [
17           {
18             "capacity_reservation_preference": "open",
19             "capacity_reservation_target": []
20           }
21         ],
22         "cpu_core_count": 1,
23         "cpu_options": [
```



Terraform State



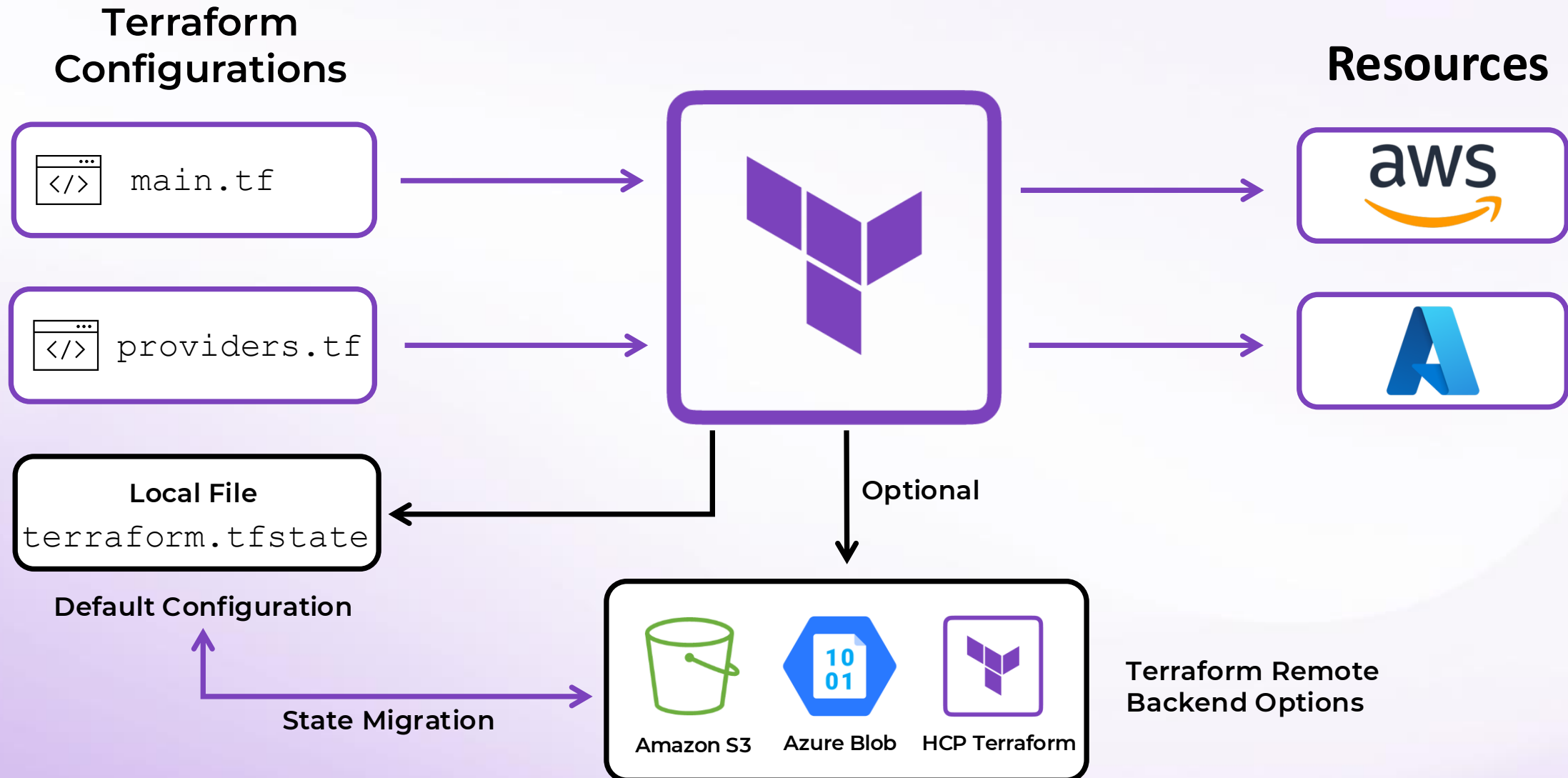


State Storage

- ✧ By default, Terraform stores state in the current working directory in a JSON-formatted file named `terraform.tfstate`
- ✧ Terraform needs to "lock" the state file when performing any operations that might write to the file (to avoid conflicts)
- ✧ Terraform can be configured to store state in a shared location. A backend configuration will determine where state is stored

```
1  {
2    "module": "module.nomad-client[\"nomad-client-3\"]",
3    "mode": "managed",
4    "type": "aws_instance",
5    "name": "this",
6    "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
7    "instances": [
8      {
9        "index_key": 0,
10       "schema_version": 1,
11       "attributes": {
12         "ami": "ami-03a6eaae9938c858c",
13         "arn": "arn:aws:ec2:us-east-1:634211456147:instance/i-0eb4591447d4fdf7c",
14         "associate_public_ip_address": true,
15         "availability_zone": "us-east-1c",
16         "capacity_reservation_specification": [
17           {
18             "capacity_reservation_preference": "open",
19             "capacity_reservation_target": []
20           }
21         ],
22         "cpu_core_count": 1,
23         "cpu_options": [
```

Terraform – Remote State





Remote State Configuration



```
1 terraform {  
2   backend "type" {  
3     # Backend-specific configuration  
4   }  
5 }
```

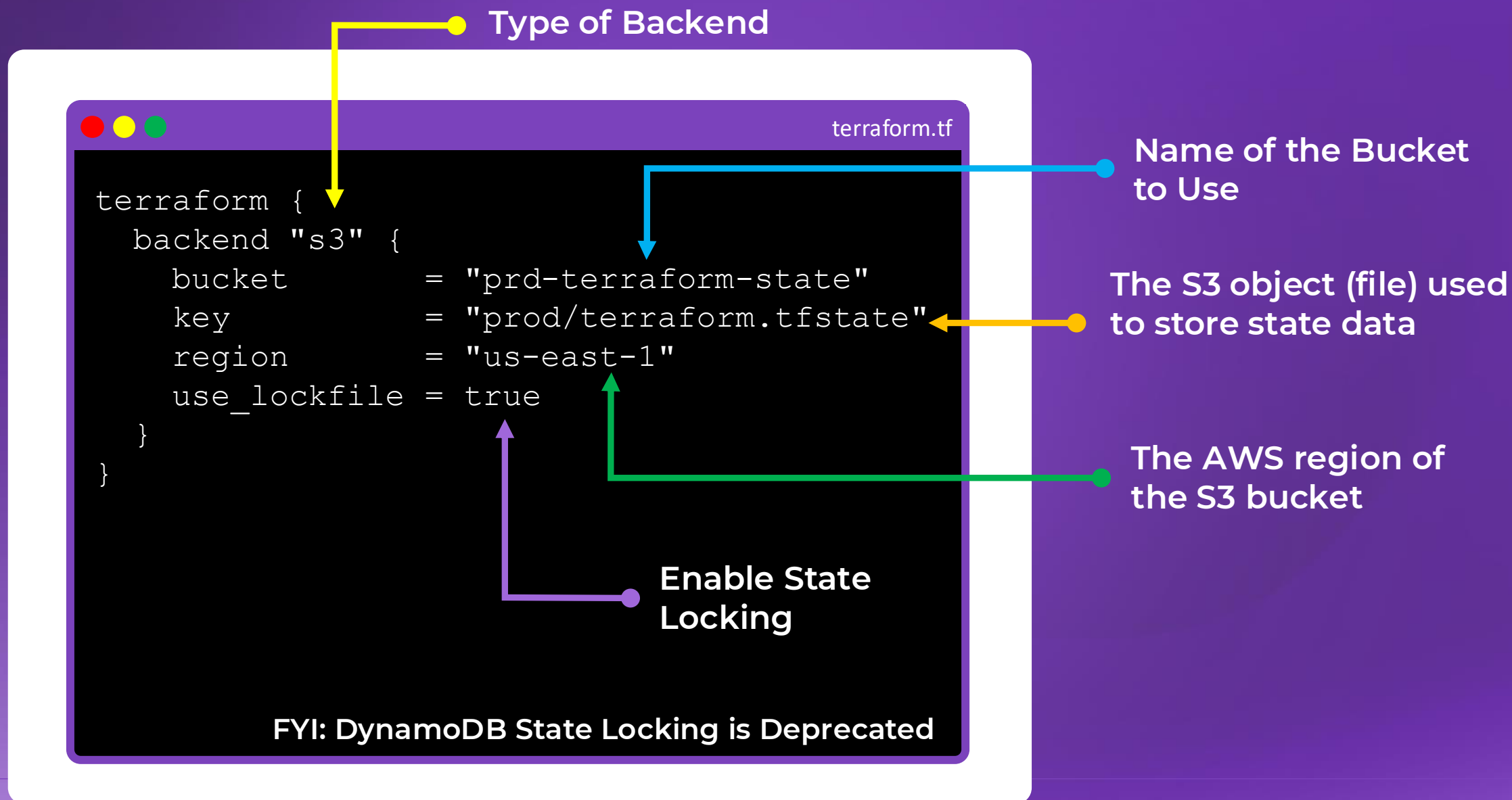
Key Components

- ❧ **Backend Type** - Specifies where state is stored (s3, azurerm, gcs, etc.)
- ❧ **Backend Configuration** - Provider-specific settings for accessing the backend
- ❧ **State Locking** - Optional but recommended (Lock file for S3, automatic for others)

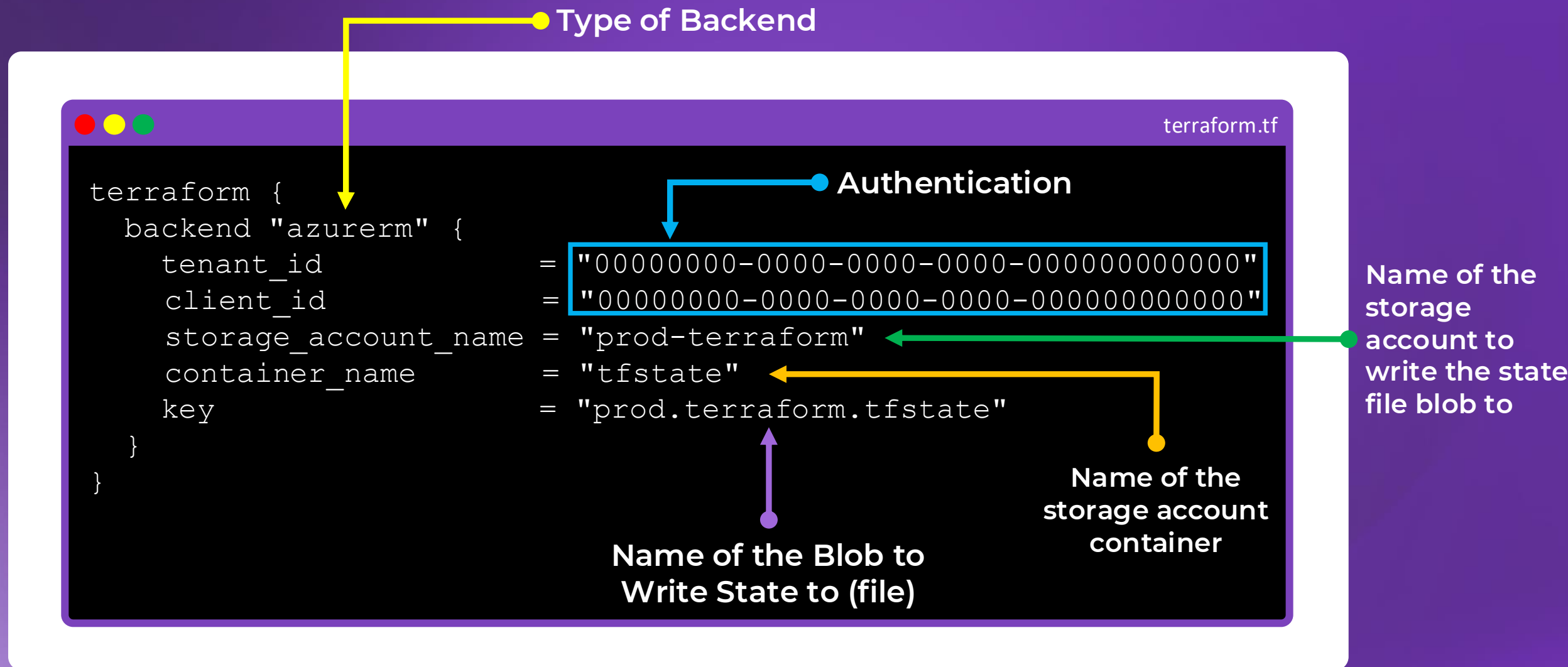
Important Notes

- ❧ Backend configuration goes in a `terraform` block, not a `provider` block
- ❧ You must run `terraform init` after adding or changing backend configuration

Remote State – Amazon S3

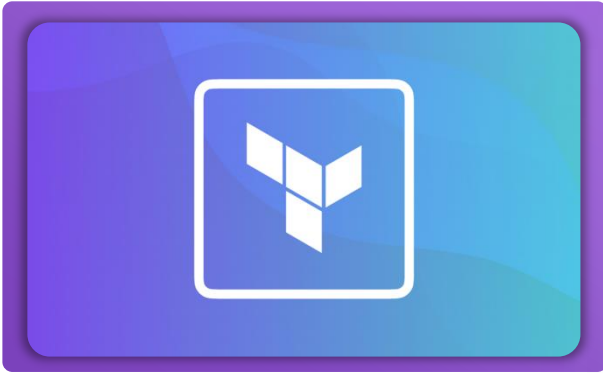


Remote State – Azure Blob Storage



Do NOT set your authentication in this file – use roles or environment variables instead

State Storage Best Practices



Local State

Local State is the default, storing `terraform.tfstate` in your working directory. Perfect for learning and solo projects.

Limitations:

- ✖ no team collaboration
- ✖ no locking
- ✖ plain text security risks
- ✖ no automatic backups



Remote State

- ✔ Remote State uses backends like S3 or Azure Storage for centralized storage.
- ✔ Provides team collaboration, state locking, encrypted storage, and automatic versioning.
- ✔ Requires initial setup and may incur costs.



Best Practices

- ✔ Use local state for learning and experimentation only.
- ✔ Use remote state with locking for all team projects and production.
- ✔ This is the recommended approach for the exam and industry standard.



Why Inspect Terraform State?

Understanding What Terraform "Knows"

You've created infrastructure with Terraform. Now what?

Sometimes you might need to:

- ✧ See what resources Terraform is managing
- ✧ Check the current recorded state of a resource
- ✧ Troubleshoot why a plan shows unexpected changes
- ✧ Verify what Terraform "thinks" exists

Inspecting State using the Terraform CLI

Primary CLI Commands:

- ✧ `terraform state list` – list all resources in Terraform state
- ✧ `terraform show` – display state or plan details

Other subcommands available:

- ✧ `terraform state mv` – move a resource
- ✧ `terraform state rm` – remove a resource from state
- ✧ `terraform state pull` – download state from current location
- ✧ `terraform state push` – push state to the configured backend

Important:

- ✓ Most have been replaced by config-driven blocks (moved, removed)
- ✓ They're not emphasized on the 004 exam



Listing Resources

```
terraform state list
```



Quick Inventory of Managed Resources

See everything that Terraform is Managing or check if a resource is already managed



Find the Exact Address of a Resource

Maybe you need the address for other commands or for troubleshooting



Verify a Resource

Make sure the resource was successfully created and is being managed.



TERMINAL

```
$ terraform state list
```

```
data.aws_availability_zones.available  
data.aws_region.current  
aws_eip.nat_gateway_eip  
aws_internet_gateway.internet_gateway  
aws_nat_gateway.nat_gateway  
aws_route_table.private_route_table  
aws_route_table.public_route_table  
aws_subnet.public[0]  
aws_subnet.public[1]  
aws_vpc.vpc
```

Only returns the address of resources



terraform show

Displays human-readable output of your state or plan file

When to use it:

- See all attributes of your resources
- Review planned changes in detail
- Inspect computed values and metadata
- Understand the current recorded state



TERMINAL

```
$ terraform show
# aws_eip.nat_gateway_eip:
resource "aws_eip" "nat_gateway_eip" {
  allocation_id      = "eipalloc-077b7c1af359aa7"
  association_id     = null
  carrier_ip         = null
  customer_owned_ip  = null
  customer_owned_ipv4_pool = null
  domain             = "vpc"
  id                 = "eipalloc-077b7c1af359a6e"
  instance           = null
  network_border_group = "us-east-1"
  network_interface  = null
  private_ip         = null
  public_dns         = "ec2-34-197-203-182....."
  public_ip          = "34.197.203.182"
  public_ipv4_pool    = "amazon"
  tags               = {
    "Name" = "demo_igw_eip"
  }
  tags_all           = {
    "Name" = "demo_igw_eip"
  }
  vpc                 = true
}

# aws_internet_gateway.internet_gateway:
```



What If You Need a Bit More Focus?

Want details about just one resource? Use `terraform state show <resource>`

```

$ terraform state show aws_vpc.vpc
# aws_vpc.vpc:
resource "aws_vpc" "vpc" {
  arn                                = "arn:aws:ec2:us-east-1:1234:vpc/vpc-07462485"
  assign_generated_ipv6_cidr_block = false
  cidr_block                        = "10.1.0.0/16"
  default_network_acl_id           = "acl-01b3609f712b682d0"
  default_route_table_id           = "rtb-0be178d4fc093ed83"
  default_security_group_id        = "sg-07996016eacf3b0c6"
  dhcp_options_id                  = "dopt-019aeb64"
  enable_classiclink                = false
  enable_classiclink_dns_support    = false
  enable_dns_hostnames              = false
}
```

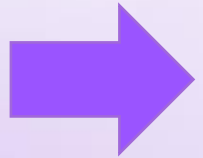
Output Abbreviated

Pro Tip: Use `terraform state list` to get exact address, then `terraform state show` to inspect it

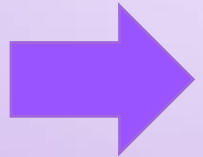
Review Legacy State Commands



- ❏ `terraform state mv` → **moved block**
- ❏ `terraform state rm` → **removed block**
- ❏ `terraform state pull/push` → **Manual state operations (rare)**



HashiCorp is moving toward "configuration as code" for all state operations - it's version-controlled, reviewable, and collaborative.



For the 004 exam: Know these exist, but focus on the config-driven alternatives



I've Detected State Drift – How Do I Fix It?

01

Revert the Changes

Force infrastructure back to match your configuration file using a regular `terraform apply`.

Your `.tf` files are the source of truth - overwrite the real-world changes.

02

Accept the Changes

Update state to match reality using **Refresh-Only Mode**.

The real infrastructure becomes the source of truth - update state to accept it.

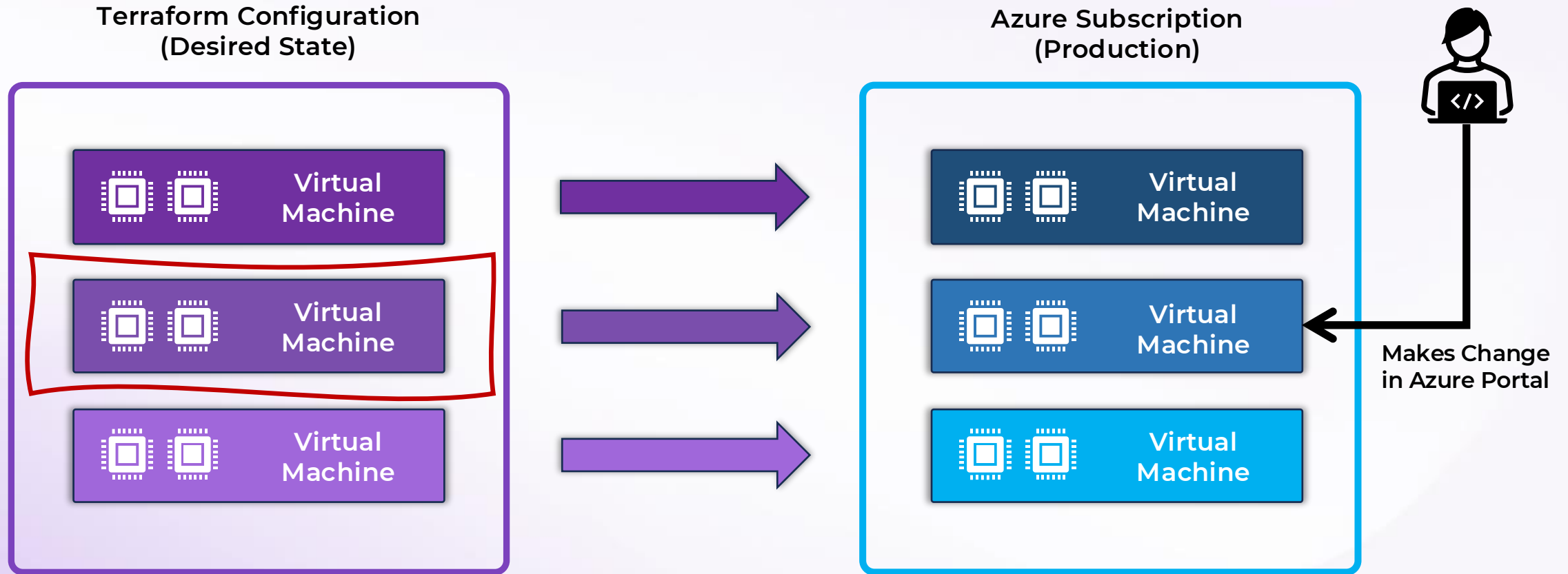
DECIDE

Which Should You Choose?

Revert it if the manual change was a mistake or unauthorized.

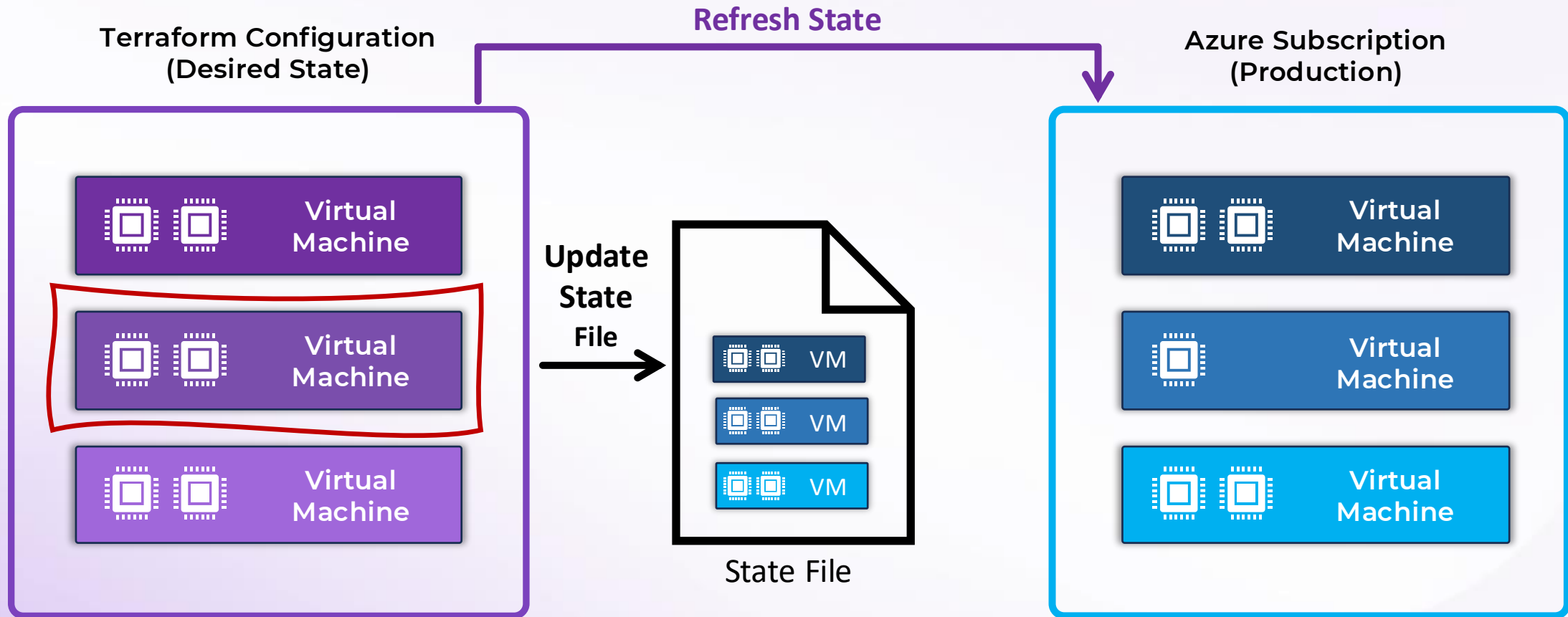
Accept it if the change is valid and you want to keep it.

Revert the Changes



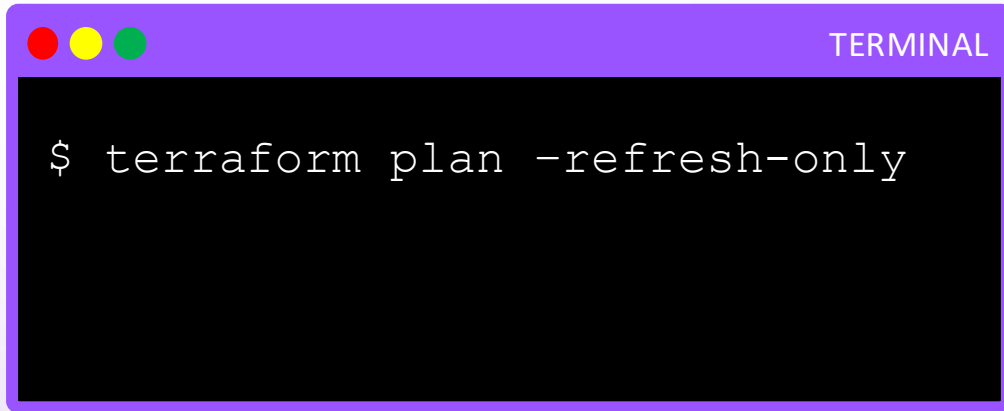
Use `terraform apply` to revert the changes
Because you're desired configuration hasn't changed

Accept the Changes using Refresh-Only Mode



Use `terraform apply -refresh-only` to update state to match changes
Reconcile your state file to match the drifted remote objects

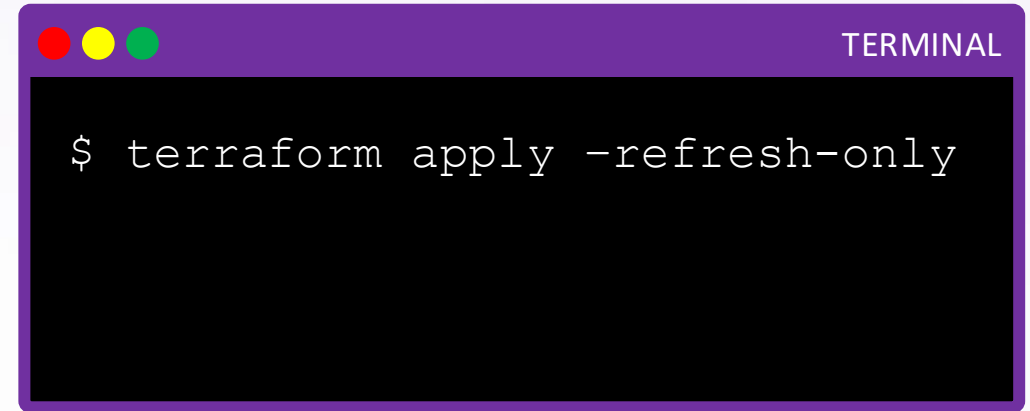
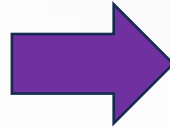
Refresh-Only Mode



```
TERMINAL
$ terraform plan -refresh-only
```

Preview Only - No Changes Made

- ❧ Shows you what **WOULD** change in state
- ❧ Safe - makes no modifications
- ❧ Use this to investigate drift before committing



```
TERMINAL
$ terraform apply -refresh-only
```

Actually Updates State File

- ❧ Actually updates the state file
- ❧ Makes the changes permanent
- ❧ Use after you've reviewed and decided to accept



State Refactoring

Reorganizing how Terraform tracks your infrastructure without changing the infrastructure itself

Common scenarios:

- ✚ You renamed a resource in your code
- ✚ You're reorganizing modules
- ✚ You need to adopt existing infrastructure
- ✚ You want to stop managing something but keep it running

Without refactoring tools, Terraform thinks you're destroying old resources and creating new ones





The Old Way

vs

The New Way

- ❯ Used separate CLI commands (terraform state mv, terraform state rm, terraform import)
- ❯ Commands are run outside your configuration
- ❯ Not version-controlled
- ❯ Hard to review in pull requests
- ❯ Risky - one wrong command and you've modified state permanently

- ❯ Use configuration blocks (moved, removed, import)
- ❯ Declared in your `.tf` files
- ❯ Version-controlled alongside your infrastructure
- ❯ Reviewable in PRs
- ❯ Part of the standard `plan/apply` workflow
- ❯ Can be tested before being applied



Core Refactoring Blocks

moved block

```
1 moved {  
2   from = azurerm_subnet.subnet1  
3   to   = azurerm_subnet.prod_private  
4 }
```

Purpose

Rename or relocate resources in state

When to Use

Refactoring code structure, renaming resources

removed block

```
1 removed {  
2   from = aws_instance.production_db  
3  
4   lifecycle {  
5     destroy = true  
6   }  
7 }
```

Purpose

Stop managing a resource but keep it running

When to Use

Handing off ownership, removing from Terraform control

import block

```
1 import {  
2   to = aws_s3_bucket.terraform_state  
3   id = "terraform_state_bucket"  
4 }
```

Purpose

Adopt existing infrastructure into Terraform

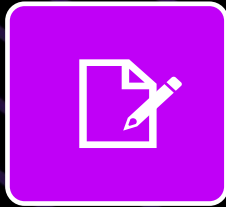
When to Use

Bringing manually-created resources under management



The Refactoring Workflow

Write



Add blocks to
Terraform config

Plan



terraform
plan
(validate changes)

Apply



terraform
apply
(updates state)

LECTURE

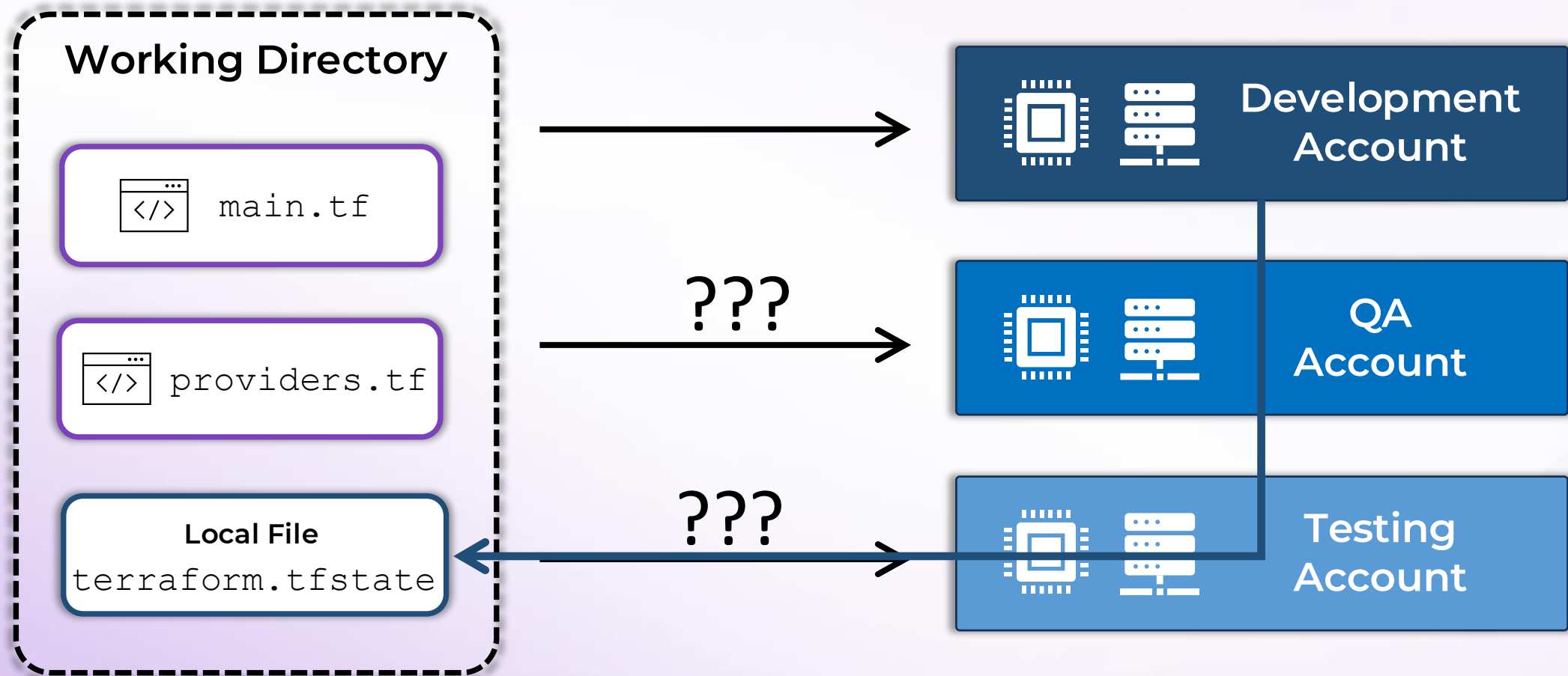
Understanding Terraform Workspaces



Scenario: Managing Multiple Environments



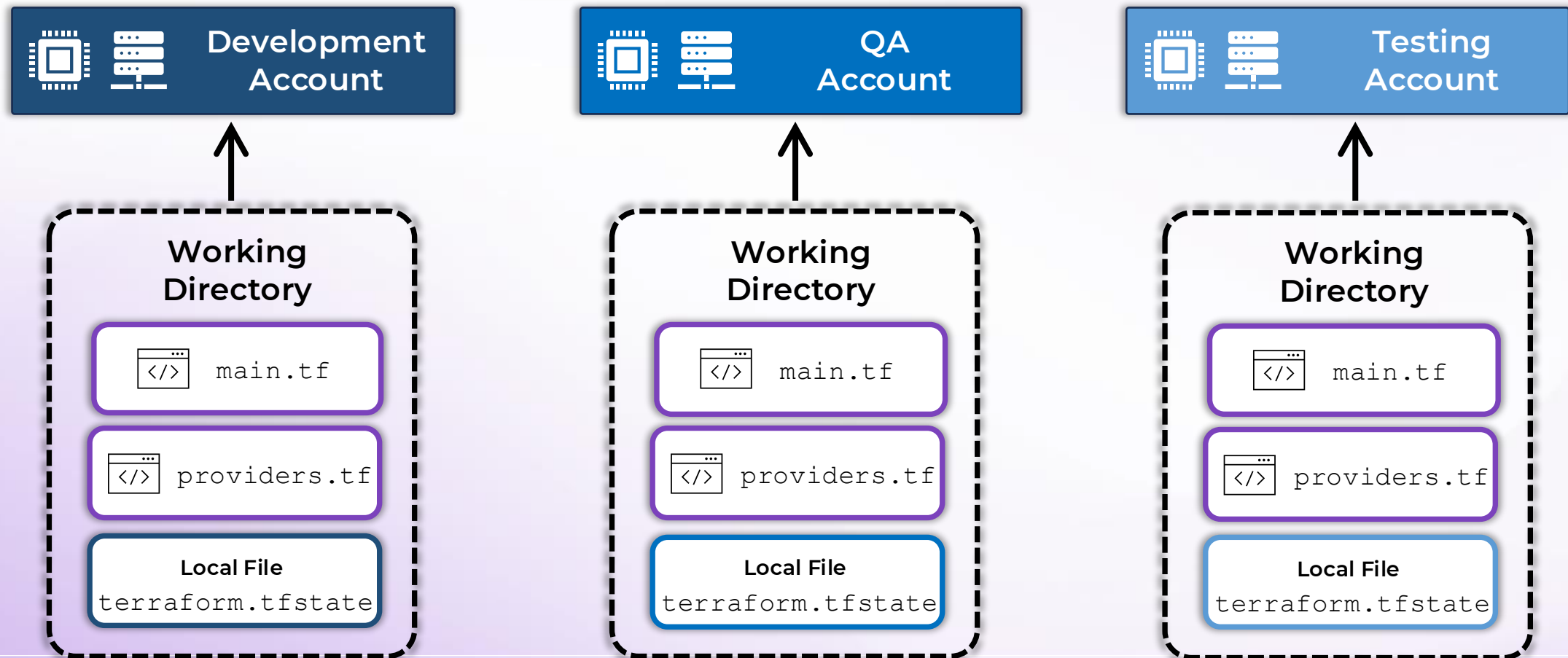
How do you manage three separate versions of the same configuration?



Scenario: Managing Multiple Environments



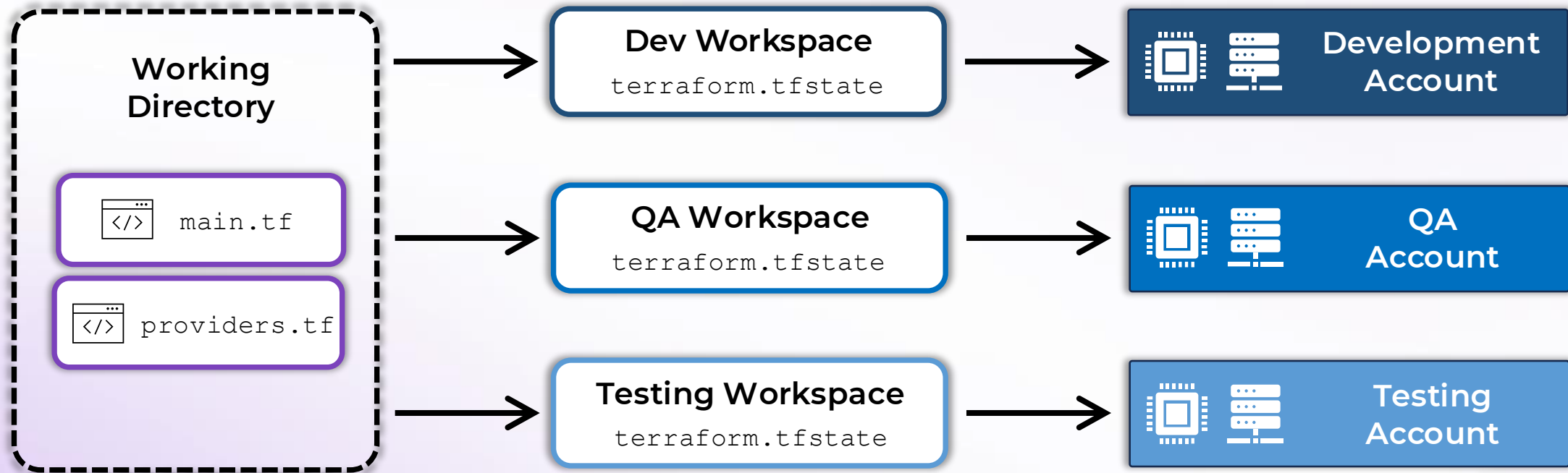
Create Multiple Working Directories (Results in Code Duplication)



Scenario: Managing Multiple Environments



Use Terraform Workspaces



- ✓ Same configuration code across all environments
- ✓ Each workspace maintains its own isolated state
- ✓ No code duplication or manual syncing required



What is a Terraform Workspace?

A workspace is a named container for a separate instance of your Terraform state.



Each workspace maintains its own isolated state file

Each workspace tracks different infrastructure resources independently



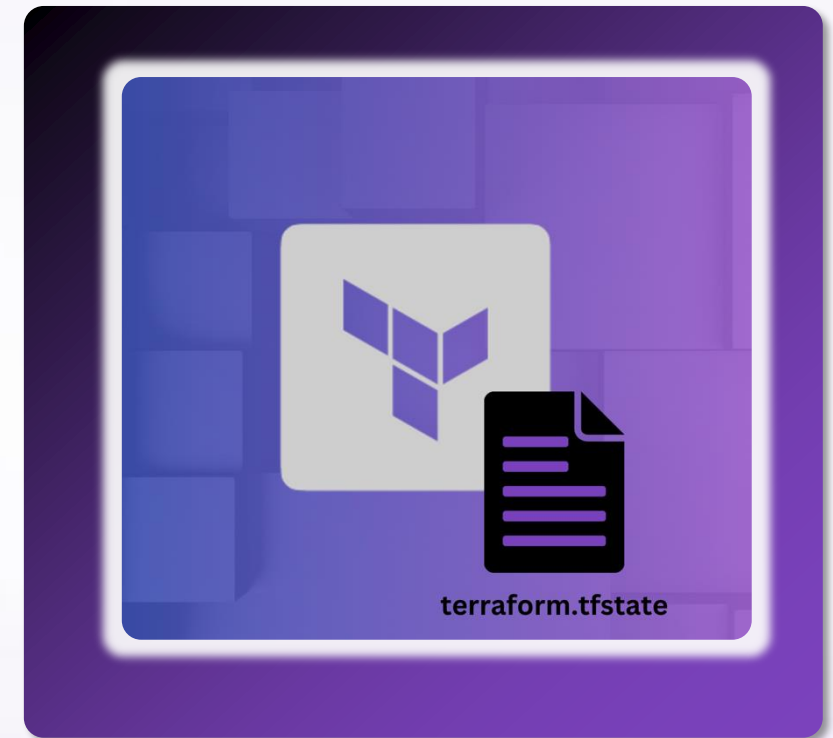
You've already been using one!

Terraform automatically creates a workspace called `default` when you run `terraform init`

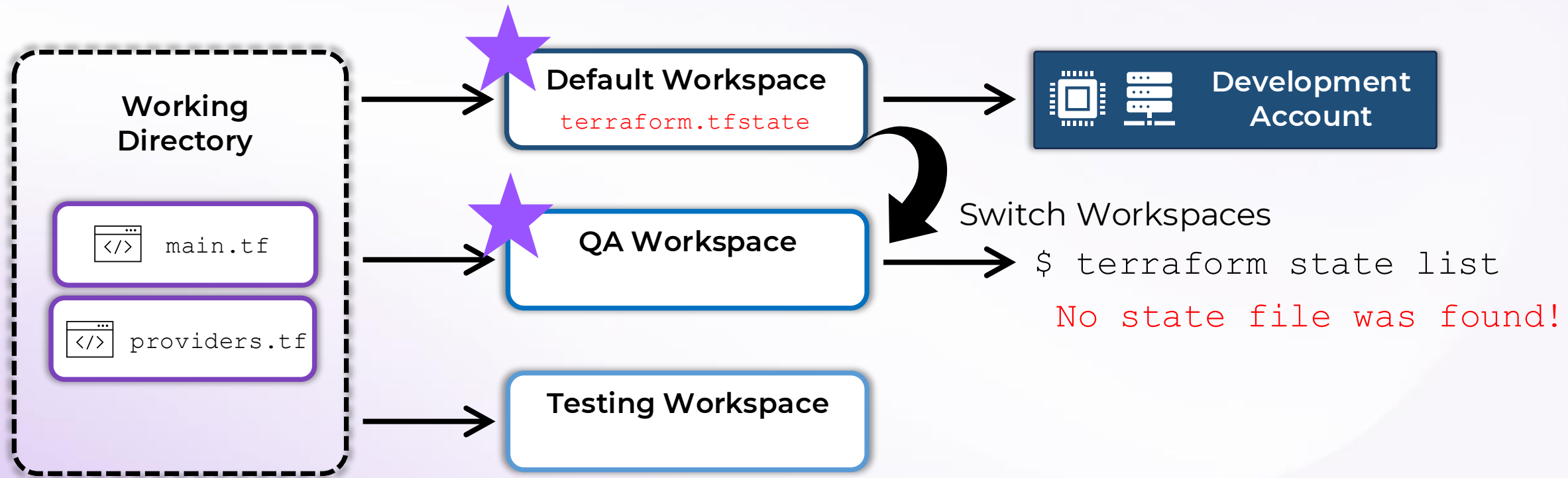


Switch workspaces to switch environments

Same configuration code, same backend, but different state for dev, staging, and production



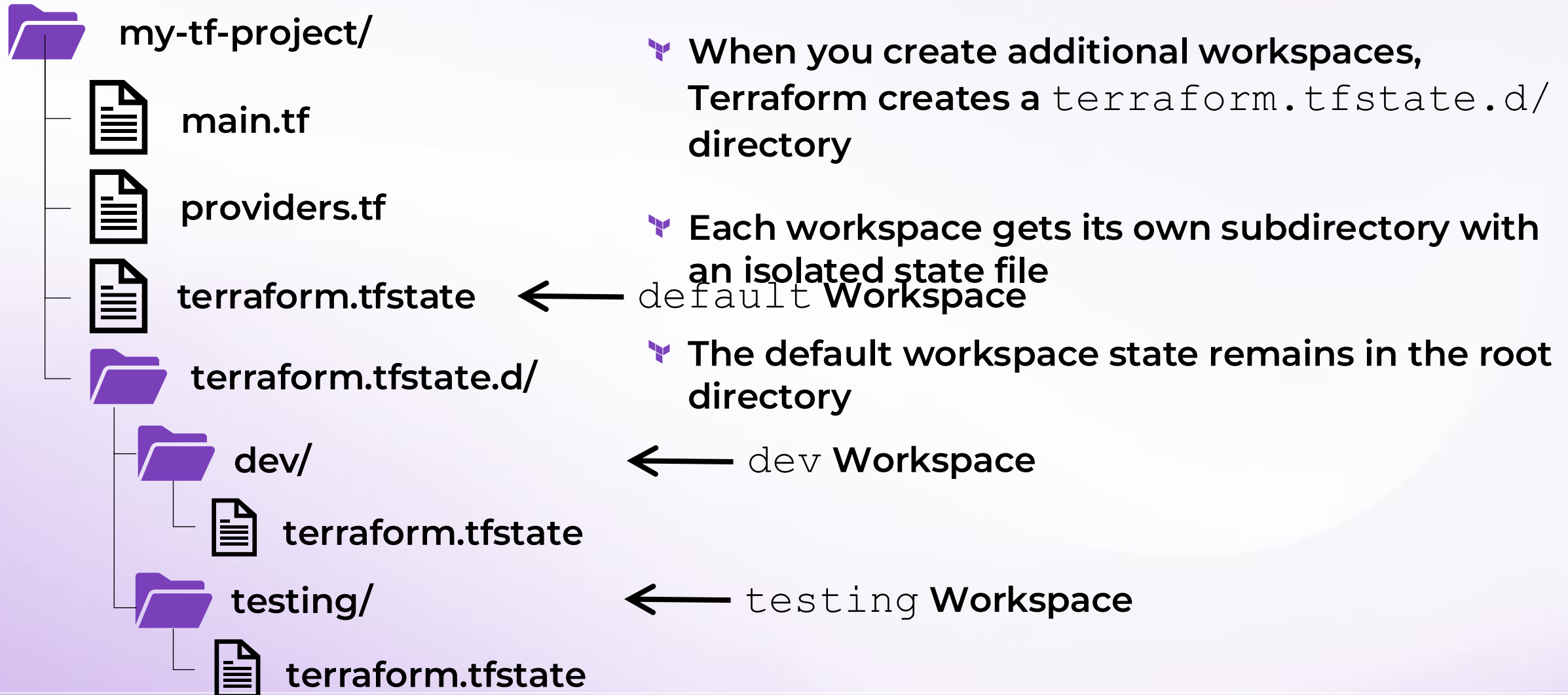
Working with Multiple Workspaces



Each workspace maintains its own isolated state file

Each workspace tracks different infrastructure resources independently

How Workspaces Store State Files



Creating and Switching Workspaces



The `terraform workspace` command

- `list` → List all workspaces
- `new` → Create a new workspace
- `select` → Change the selected namespace
- `show` → Display the current workspace
- `delete` → Delete the specified workspace

```

$ terraform workspace list
default
dev
prod
* qa
test

```

A terminal window with a purple title bar and three colored window control buttons (red, yellow, green) on the left. The title bar is labeled 'TERMINAL'. The terminal shows the command `$ terraform workspace list` and its output: `default`, `dev`, `prod`, `* qa`, and `test`. A purple arrow points from the `* qa` line to the text 'Currently selected workspace' below the terminal.

```

$ terraform workspace new dev
Created and switched to workspace "dev"!

```

A terminal window with a purple title bar and three colored window control buttons (red, yellow, green) on the left. The title bar is labeled 'TERMINAL'. The terminal shows the command `$ terraform workspace new dev` and its output: `Created and switched to workspace "dev"!`. A purple arrow points from the `new` command in the list above to this terminal window.

Currently selected
workspace

Two Types of Workspaces: Don't Get Them Confused!



VS

CLI Workspaces

(What we're learning in this lecture)

- ❧ Multiple state files in **one configuration**
- ❧ Switch between environments (dev, staging, prod)
- ❧ Same backend, same code
- ❧ Lives on your **local machine** or remote backend
- ❧ `terraform workspace select dev`

HCP Terraform Workspaces

(HCP Terraform)

- ❧ A complete working directory in the cloud
- ❧ Each workspace has its own configuration, variables, and state
- ❧ Different workspaces can be connected to different repositories
- ❧ Includes VCS integration, run history, and team collaboration
- ❧ Managed through the HCP Terraform UI or API